

ProbeAI: an AI tester whose bug reports you can check

Yuanhang Wang · Mechanical Engineering PhD candidate, AUT · October 2026

1. What I understood the problem to be

Small teams ship fast, and testing is the first thing cut. Writing automated tests is a project in itself, and manual click-through is slow and inconsistent. An AI tester could remove that set-up cost, but the brief asks the hard question itself: *how do you handle false positives?* A tester that reports non-bugs gets ignored. In one published study (WebProber, 120 websites), 85% of an AI tester's reported bugs were false alarms, mostly because the browser automation failed, not because the AI reasoned badly. So I treated the real problem as **trust, not coverage**: not "can an AI click around an app?" but "how does a developer know a reported bug is real?"

2. Options I considered, and why I chose this one

Connect. A URL, the source code, or an upload. I chose a URL: every team has a staging address, and it needs no code access and nothing written first.

Find bugs. (a) AI writes Playwright scripts: gives regression tests, not checked bug reports. (b) A free-roaming agent: hard to reproduce and to show live. (c) Build on Stagehand: a fast-moving dependency with hidden AI calls. (d) Buy a tool such as QA.tech or Momentic: often right for a client, and building one taught me what to ask.

My choice: split the job. The AI decides *what to test*: it plans up to five user "missions" (live mode runs the first three) and picks one browser action at a time, writing down beforehand what should happen. The browser, not the AI, records *what happened* (every request, error and screenshot). Anything suspicious is replayed from a clean start without the AI. Tiers are set by rule: **Confirmed** = hard browser evidence (for example a server error) that comes back in a clean replay; **Likely** = weaker evidence, or the AI's judgement alone, which can never reach Confirmed by itself; **Improvement** = a suggestion, never called a defect; **Dropped** = looked at and dismissed, with the reason shown. Each finding has steps, screenshots, expected versus observed.

3. How I checked it

I built a small to-do app with six planted faults that logs when each one really fires (the answer key), plus a clean copy as a control. In 5 baseline test runs the failing delete and the blank-title bug were found every time. The "items left" counter went wrong in every run and was reported in none, because the AI was only shown what *changed* on the page. I changed the evidence it sees, not its instructions, and it was reported in 3 of 3 later runs. Three faults were never found; in the baseline, two were never proposed and one ran out of steps. On the clean copy: 1 false alarm in 5 runs, then 0 in 3. Every finding was read one by one; with samples this small I report counts, not percentages. A live run takes about a minute and costs about five US cents. When the AI kept repeating a failed click, I added a rule from browser evidence instead of another instruction: same button, same server error, twice, stop that check.

4. What I left out of scope, and why

Reading the code; logins (it would hold passwords; my safety rule blocks password fields); accessibility, speed, mobile and visual-regression scans (good tools exist, different problem); large-site crawling; production sites (staging only, one site, and it will not press pay, publish, upload, download or delete-account); a dashboard or database (JSON files and one page are enough). **Known limits:** I tested at length only on my own app, whose faults I know; replay needs an app that can be reset; live mode runs only the first three planned checks to stay near a minute (a speed-versus-coverage trade-off), so a live run can end with Likely findings only; and the Improvement tier is my least validated part: of the 14 suggestions in my recordings I would show none.

5. How I used AI

Twice. *Inside the tool*, the AI does the part that needs judgement and is never trusted with the evidence. *To build it*, I used a review loop instead of trusting any one model: Claude Opus made and reviewed the design decisions, Claude Sonnet wrote the code, and at key decision points I took the plan to ChatGPT as an outside auditor. Opus then ruled on each ChatGPT point, adopting some and rejecting others with a written reason. I did the final check by hand. Decisions are written down before the code, with a daily work log and one git commit per task, so any later session could pick up from the record, including the times an assistant was wrong and a re-check caught it. The limit: the code was written and reviewed within one model family, so re-run it rather than trust it.